

T2T System Logic Explanation

Based on: `arduino/bottle_sensor/bottle_sensor.ino`, `arduino/qr_arduino/qr_arduino.ino`, and `arduino/bridge.py`

1. System Roles

Arduino #1 (bottle_sensor): LCD user interface, ultrasonic bottle detection, gate servo control, buzzer feedback.

Arduino #2 (qr_arduino): QR input, IR obstacle alerts, capacitive material scan (PET/GLASS/METAL).

bridge.py: Central coordinator between both Arduinos + Firebase lookup/update logic.

2. Communication Protocol

Bridge to bottle_sensor

- `STUDENT:<name>:<id>` starts session
- `ACCEPTED` opens gate + increments bottle count on bottle_sensor side
- `REJECTED:<reason>` keeps gate closed + shows rejection
- `OBSTACLE:1` or `OBSTACLE:0` shows IR obstruction state
- `ERROR:notfound` shows student-not-found message

bottle_sensor to Bridge

- `BOTTLE_DETECTED:<studentID>` when ultrasonic edge-detects bottle insertion
- `SESSION_START:<studentID>` and `SESSION_END:<studentID>`

Bridge and qr_arduino

- `QR:<uid>` from qr_arduino after scan
- `SCAN_MATERIAL` sent by bridge to request capacitive reading
- `MATERIAL:PET|GLASS|METAL` response from qr_arduino
- `IR:ALERT` and `IR:CLEAR` from qr_arduino

3. End-to-End Flow (Normal Case)

- 1) Student scans QR on qr_arduino
- 2) qr_arduino sends QR:<uid> to bridge
- 3) bridge finds student in Firebase
- 4) bridge sends STUDENT:<name>:<docID> to bottle_sensor
- 5) bottle_sensor enters active session and shows welcome LCD
- 6) user inserts bottle, ultrasonic detects leading edge
- 7) bottle_sensor sends BOTTLE_DETECTED:<id>
- 8) bridge sends SCAN_MATERIAL to qr_arduino
- 9) qr_arduino classifies material and sends MATERIAL:PET/GLASS/METAL
- 10) bridge decides:
 - PET -> sends ACCEPTED and updates Firebase points/bottles
 - OTHER -> sends REJECTED:<type>
- 11) bottle_sensor updates LCD + buzzer + gate state
- 12) session continues until timeout/max bottles/end command

4. bottle_sensor State Logic

Key state variables

- `sessionActive` : allows bottle checks only during a student session
- `waitingForResult` : lockout after sending `BOTTLE_DETECTED` ; prevents duplicate detections until result arrives
- `bottleWasPresent` : edge detection (fire only when bottle first appears)
- `obstacleBlocked` : pause detection when IR alert is active
- `screenRevertAt` : non-blocking LCD temporary screen timer
- `gateOpen` and `gateOpenedAt` : servo auto-close timing

Main loop behavior

1. Read incoming bridge command.
2. Handle timed screen reversion (without blocking entire loop).
3. Auto-close servo gate after configured open duration.
4. If session is active and not blocked and not waiting for classification result, run ultrasonic bottle detection.
5. Print periodic distance debug line.

Why edge-trigger + cooldown matters

Ultrasonic measurements can remain in-range while bottle stays near the sensor. The combination of `bottleWasPresent` (edge-trigger) and `BOTTLE_COOLDOWN` avoids repeated `BOTTLE_DETECTED` spam for one physical insertion event.

5. qr_arduino Material Logic

When `SCAN_MATERIAL` is received, the board samples the digital capacitive output repeatedly (40 samples total) and counts how many reads are LOW. The result bucket determines material type:

- Low count: PET
- Mid count: GLASS
- High count: METAL

This classification is intentionally performed on qr_arduino, while the final accept/reject action is decided by bridge.py.

6. bridge.py Decision Logic

1. Reads events from both serial ports continuously.
2. On QR event, looks up student and sends session-start payload to bottle_sensor.
3. On bottle detection, requests material scan and waits for response.
4. On material result:
 - PET: send `ACCEPTED`, update Firebase points/bottles.
 - GLASS/METAL/UNKNOWN: send `REJECTED:<reason>`.
5. If Firebase is unavailable, enqueue unsent updates in local backup and retry periodically.

7. Error and Safety Paths

- **Student not found:** bridge sends `ERROR:notfound`; bottle_sensor shows error screen and returns to idle.
- **Material timeout:** if no material response arrives in time, bridge sends rejection timeout.
- **Obstacle active:** IR alerts pause bottle acceptance path and display warning.
- **Session timeout:** bottle_sensor auto-ends inactive sessions after configured duration.

- **Gate fail-safe:** gate is explicitly closed on rejection, timeout, and session end.

8. Practical Summary

The architecture separates sensing, actuator/UI control, and business logic. That separation makes the system easier to debug and safer: `qr_arduino` specializes in signal acquisition, `bottle_sensor` specializes in user-facing control and physical gate behavior, and `bridge.py` enforces the final decision and data consistency with Firebase/offline backup.

Generated on April 23, 2026.